

Tipos de dados

Programação Funcional

Marco A L Barbosa

malbarbo.pro.br

Departamento de Informática

Universidade Estadual de Maringá



Este trabalho está licenciado com uma Licença Creative Commons - Atribuição-Compartilhagual 4.0 Internacional.

<http://github.com/malbarbo/na-progfun>

Introdução

Qual é a segunda etapa no processo de projeto de funções? Definição de tipos de dados.

Qual é o propósito dessa etapa? Identificar as informações e definir como elas serão representadas.

Essa etapa pode ter parecido, até então, muito simples ou talvez até desnecessária, porque as informações que precisávamos representar eram “simples”.

No entanto, essa etapa é muito importante no projeto de programas; de fato, vamos ver que, para muitos casos, os tipos de dados vão guiar o restante das etapas do projeto.

Vamos começar com a definição do que é um tipo de dado.

Um **tipo de dado** é um conjunto de valores que uma variável pode assumir.

Cada valor de um tipo de dado também é chamado de uma **instância** do tipo.

Exemplos

- $\text{Booleano} = \{\text{verdadeiro}, \text{falso}\}$
- $\text{Natural} = \{0, 1, 2, \dots\}$
- $\text{Inteiro} = \{\dots, -2, -1, 0, 1, 2, \dots\}$
- $\text{String} = \{ '', 'a', 'b', \dots \}$
- String que começa com a = $\{ 'a', 'aa', 'ab', \dots \}$

Durante a etapa de definição de tipos de dados, identificamos as informações e definimos como elas são representadas no programa.

Como determinar se um tipo de dado **é adequado** para representar uma informação?

Um inteiro é adequado para representar a quantidade de pessoas em um planeta?

- Não, pois ele pode ser negativo, mas a quantidade de pessoas não. Ou seja, o tipo permite representar valores inválidos.

E um natural de 32 bits?

- Não, pois o valor máximo que ele pode representar é 4.294.967.295, e o planeta Terra tem mais pessoas que isso. Ou seja, o tipo não permite representar todos os valores válidos.

E um natural?

- Sim, é adequado. Cada valor do conjunto dos números naturais representa uma quantidade válida de pessoas, e cada possível quantidade de pessoas pode ser representada por um número natural.

Diretrizes para o projeto de tipos de dados:

- Torne os valores válidos representáveis.
- Torne os valores inválidos irrepresentáveis.

Vamos aplicar essas diretrizes a uma série de exemplos.

No exemplo da escolha do combustível, nós definimos os seguintes tipos:

```
/// O preço do litro do combustível. Requer que seja um número positivo.
```

```
type Preco = Float
```

```
/// O tipo do combustível. Requer que seja "Álcool" ou "Gasolina".
```

```
type Combustivel = String
```

Esses tipos estão de acordo com as diretrizes para o projeto de tipos de dados?

Não!

Vamos resolver essa questão começando com `Combustivel`; para o `Preco` vamos precisar de uma ferramenta que veremos no capítulo **Funções totais**.

Enumerações

Em um **tipo enumerado**, todos os valores do tipo são enumerados explicitamente.

A forma geral para definir tipos enumerados é:

```
[pub] type NomeDoTipo {  
    Valor1  
    Valor2  
    ...  
}
```

Cada tipo enumerado pode ter 0 ou mais valores. O nome e os valores do tipo devem começar com letra maiúscula.

Quando usar tipos enumerados?

Quando todos os valores válidos para o tipo podem ser nomeados.

Vamos definir um tipo enumerado para representar o tipo combustível.

```
/// O tipo do combustível.  
type Combustivel {  
    Alcool  
    Gasolina  
}
```

Podemos utilizar os operadores `==` e `!=` e a função `string.inspect` com os valores de tipos enumerados.

```
> Alcool == Gasolina  
False  
> Alcool != Gasolina  
True  
> string.inspect(Gasolina)  
"Gasolina"
```

Assim como para valores do tipo `Bool`, podemos utilizar a expressão `case` para processar valores de tipos enumerados. De fato, o tipo `Bool` é um tipo enumerado com os valores `True` e `False` pré-definido na linguagem.

```
fn mensagem_combustivel(  
    c: Combustivel  
) -> String {  
    case c {  
        Alcool -> "Use álcool."  
        Gasolina -> "Use gasolina."  
    }  
}
```

A análise dos casos precisa ser exaustiva.

```
fn mensagem_combustivel(c: Combustivel) -> String {  
  case c {  
    Alcool -> "Use álcool."  
  }  
}
```

error: Inexhaustive patterns

```
└─ src/arquivo.gleam:2:3  
2 |  
3 | ┌ case c {  
4 | │   Alcool -> "Use álcool."  
   │   }  
   └─ ^
```

This case expression does not have a pattern for all possible values. If it is run on one of the values without a pattern then it will crash.

The missing patterns are:

Gasolina

Com o tipo enumerado, a função do capítulo anterior passa a produzir um combustível de verdade:

```
/// Encontra o combustível que deve ser utilizado no abastecimento. Produz
/// Alcool se *preco_alcool* for menor ou igual a 70% do *preco_gasolina*,
/// produz Gasolina caso contrário.
fn seleciona_combustivel(
    preco_alcool: Preco,
    preco_gasolina: Preco,
) -> Combustivel {
    case preco_alcool <=. 0.7 *. preco_gasolina {
        True -> Alcool
        False -> Gasolina
    }
}
```

Produzir um combustível inválido deixou de ser possível: "Alcool" agora é erro de compilação. A `mensagem_combustivel` é quem converte o valor em texto para o usuário.

O RU da UEM cobra um valor por tíquete que depende da relação do usuário com a universidade. Para alunos e servidores que recebem até 3 salários mínimos, o tíquete custa R\$ 5,00; para servidores que recebem mais de 3 salários mínimos e docentes, R\$ 10,00; e para pessoas da comunidade externa, R\$ 19,00. Como parte de um sistema de cobrança, você deve projetar uma função que determine quanto deve ser cobrado de um usuário por uma quantidade de tíquetes.

Análise

- Determinar quanto deve ser cobrado de um usuário por uma quantidade de tíquetes.
- O usuário pode ser aluno ou servidor (até 3 s.m.) — R\$ 5; servidor (mais de 3 s.m.) ou docente — R\$ 10; ou externo — R\$ 19.

Definição de tipos de dados

- As informações são a quantidade, o tipo de usuário e o valor que deve ser cobrado.

Como representar um tipo de usuário? Criando um tipo enumerado com os valores possíveis para o tipo.

```
/// O tipo de usuário do RU da UEM.  
type Usuario {  
    Aluno  
    // Servidor que recebe até 3 salários mínimos.  
    ServidorAte3  
    // Servidor que recebe mais de 3 salários mínimos.  
    ServidorMais3  
    Docente  
    Externo  
}
```

Especificação

```
/// Determina o custo de *quant* tíquetes para um usuário do tipo *usuario*.
/// O custo de um tíquete é
/// - Aluno          5,0
/// - ServidorAte3    5,0
/// - ServidorMais3   10,0
/// - Docente         10,0
/// - Externo         19,0
/// Requer que *quant* seja maior ou igual a zero.
fn custo_tiquetes(usuario: Usuario, quant: Int) -> Float
```

A restrição sobre *quant* está escrita na especificação: por enquanto, `custo_tiquetes` é uma função parcial.

Quantos exemplos são necessários para funções que processam valores de tipos enumerados?
Pelo menos um para cada valor da enumeração.

```
pub fn custo_tiquetes_examples() {  
    check.eq(custo_tiquetes(Aluno, 3), 15.0)  
    check.eq(custo_tiquetes(ServidorAte3, 2), 10.0)  
    check.eq(custo_tiquetes(ServidorMais3, 2), 20.0)  
    check.eq(custo_tiquetes(Docente, 3), 30.0)  
    check.eq(custo_tiquetes(Externo, 4), 76.0)  
}
```

Como iniciamos a implementação de uma função que processa um valor de tipo enumerado?
Criando um caso para cada valor da enumeração.

Implementação

```
fn custo_tiquetes(usuario: Usuario, quant: Int) -> Float {  
  case usuario {  
    Aluno -> todo  
    ServidorAte3 -> todo  
    ServidorMais3 -> todo  
    Docente -> todo  
    Externo -> todo  
  }  
}
```

Agora completamos o corpo considerando cada forma de resposta dos exemplos.

Implementação

```
fn custo_tiquetes(usuario: Usuario, quant: Int) -> Float {  
  case usuario {  
    Aluno -> 5.0 *. int.to_float(quant)  
    ServidorAte3 -> 5.0 *. int.to_float(quant)  
    ServidorMais3 -> 10.0 *. int.to_float(quant)  
    Docente -> 10.0 *. int.to_float(quant)  
    Externo -> 19.0 *. int.to_float(quant)  
  }  
}
```

Verificação

Running tests...

5 tests, 5 success(es), 0 failure(s) and 0 error(s).

Revisão

O que podemos melhorar?

- Juntar com | os casos que produzem o mesmo custo;
- Fatorar a multiplicação, que é comum a todos os casos;
- Tornar a função total, escolhendo uma resposta para *quant* negativo, e acrescentar um exemplo.

```
/// ... Se *quant* for negativo, devolve 0,0.  
fn custo_tiquetes(usuario: Usuario, quant: Int) -> Float {  
  case usuario {  
    Aluno | ServidorAte3 -> 5.0  
    ServidorMais3 | Docente -> 10.0  
    Externo -> 19.0  
  }  
  *. int.to_float(int.max(0, quant))  
}
```

Escolher uma resposta para as entradas que a restrição excluía é a forma mais simples de tornar uma função total; no capítulo **Funções totais** vamos ver quando ela é adequada e quais são as outras formas.

Estruturas

Os tipos de dados que vimos até agora são atômicos, isto é, não podem ser decompostos.

Agora veremos como representar dados em que dois ou mais valores devem ficar juntos:

- Registro de um aluno;
- Placar de um jogo de futebol;
- Informações de um produto.

Chamamos estes tipos de dados de **dados compostos**, **registros** ou **estruturas**.

Em uma **estrutura**, os valores do tipo são formados por um ou mais **campos**, cada um com o seu nome e o seu tipo.

Cada valor é criado aplicando o **construtor** do tipo aos valores dos campos.

A forma geral para definir um **dado composto** é:

```
[pub] type NomeDoTipo {  
    NomeDoTipo([campo1:] Tipo1, [campo2:] Tipo2, ...)  
}
```

Quando usar dados compostos?

Quando dois ou mais itens de informação juntos descrevem uma entidade.

Vamos definir uma estrutura para representar um ponto em um plano cartesiano.

Definição

```
type Ponto {  
    Ponto(x: Int, y: Int)  
}
```

Construção

```
> let p1: Ponto = Ponto(x: 3, y: 4)  
> let p2 = Ponto(8, 2)  
> p2  
Ponto(x: 8, y: 2)
```

Acesso aos campos

```
> p1.x + p1.y  
7
```

Comparação

```
> p1 == Ponto(3, 4)  
True  
> p1 != p1  
False  
> p1 != p2  
True
```

Inspeção

```
> string.inspect(p1)  
"Ponto(x: 3, y: 4)"
```

Junto com a definição de uma estrutura, também faremos a descrição do seu propósito e dos seus campos.

```
/// Um ponto no plano cartesiano.  
type Ponto {  
    // x e y são as coordenadas do ponto.  
    Ponto(x: Int, y: Int)  
}
```

Podemos consultar o valor de um campo, mas como alterar o valor de um campo? Não é possível! Lembrem-se, estamos estudando o paradigma funcional, em que não existe mudança de estado!

Em vez de modificar o campo de uma instância da estrutura, criamos uma cópia da instância com o campo alterado.

Vamos criar um ponto **p2** que é como **p1**, mas com o valor **5** para o campo **y**.

```
> let p1 = Ponto(3, 4)
> let p2 = Ponto(p1.x, 5)
> p2
Ponto(x: 3, y: 5)
```

Quais são as limitações desse método?

- Se a estrutura tem muitos campos e desejamos alterar apenas um campo, temos que especificar a cópia de todos os outros;
- Se a estrutura é alterada pela adição ou remoção de campos, então, todas as operações de “cópia” da estrutura no código devem ser alteradas.

Gleam tem uma sintaxe especial para atualização de estruturas.

```
> let p1 = Ponto(3, 4)
> let p2 = Ponto(..p1, y: 5)
> p2
Ponto(x: 3, y: 5)
```

```
> let p3 = Ponto(..p1, x: 7)
> p3
Ponto(x: 7, y: 4)
```

```
> // Podemos atualizar mais que um campo (não faz sentido nesse exemplo)
> Ponto(..p1, x: 1, y: 2)
Ponto(x: 1, y: 2)
```

Exemplo - outras linguagens

A ideia de estruturas imutáveis que são “atualizadas” através de cópias está presente em diversas linguagens. A seguir temos um exemplo em Python e outro em Rust.

```
from dataclasses import dataclass, replace

@dataclass(frozen=True)
class Ponto:
    x: int
    y: int

>>> p = Ponto(10, 20)

>>> # Atribuição inválida, p é imutável
>>> p.x = 8

>>> # Cria uma cópia de p alterando x para 8
>>> p1 = replace(p, x=8)
>>> p1
Ponto(x=8, y=20)
```

```
struct Ponto {
    x: i32,
    y: i32,
}

pub fn main() {
    let p = Ponto { x: 10, y: 20 };

    // Atribuição inválida, p é imutável
    p.x = 8;

    // Cria uma cópia de p alterando x para 8
    let p1 = Ponto {x: 8, ..p};
    assert_eq!(p1.x, 8);
    assert_eq!(p1.y, 20);
}
```


Campo minado é um famoso jogo de computador. O jogo consiste em um campo retangular de quadrados que podem ou não conter minas escondidas. Os quadrados podem ser abertos clicando sobre eles. O objetivo do jogo é abrir todos os quadrados que não têm minas. Se o jogador abrir um quadrado com uma mina, o jogo termina e o jogador perde.

Como guia para explorar o campo, cada quadrado aberto exibe o número de minas nos quadrados ao seu redor (no máximo 8). Quando um quadrado sem minas ao redor é aberto, todos os quadrados ao seu redor também são abertos. O usuário pode colocar uma bandeira sobre um quadrado fechado para sinalizar uma possível mina e impedir que ele seja aberto. Uma bandeira também pode ser removida de um quadrado.



Projete um tipo de dado para representar um quadrado em um jogo de campo minado. Não é necessário armazenar o número de bombas ao redor do quadrado pois esse valor pode ser calculado dinamicamente.

Em uma primeira tentativa poderíamos pensar: o quadrado pode ter uma mina ou não, pode estar fechado ou aberto e pode ter uma bandeira ou não. Como são três itens relacionados, então definiríamos uma estrutura. Além disso, cada item tem dois estados possíveis, então poderíamos usar booleano para representar cada estado.

```
/// Um quadrado no jogo campo minado.  
type Quadrado {  
    Quadrado(mina: Bool, aberto: Bool, bandeira: Bool)  
}
```

Nós vimos duas diretrizes para o projeto de tipos de dados:

- Torne os valores válidos representáveis.
- Torne os valores inválidos irrepresentáveis.

A definição de **Quadrado** está de acordo com essas diretrizes? Vamos verificar!

Quantas possíveis instâncias distintas existem de **Quadrado**? São três campos, cada um pode assumir dois valores, portanto, $2 \times 2 \times 2 = 8$.

Vamos listar essas instâncias e analisar se todas são válidas.

mina	aberto	bandeira	Válido?
F	F	F	Sim
F	F	V	Sim
F	V	F	Sim
F	V	V	Não
V	F	F	Sim
V	F	V	Sim
V	V	F	Sim
V	V	V	Não

Temos dois estados inválidos!

Como evitar estes estados inválidos? Primeiro temos que entender o problema.

A questão é que apenas 3 das 4 possíveis combinações dos valores dos campos

aberto e **bandeira** são válidas: aberto, fechado ou fechado com bandeira.

Para resolver a situação podemos “juntar” os campos **aberto** e **bandeira** em um campo **estado** que pode assumir um desses três valores.

```
/// O estado de um quadrado no
/// campo do jogo.
type Estado {
    Aberto
    Fechado
    FechadoComBandeira
}

/// Um quadrado no campo de jogo.
type Quadrado {
    Quadrado(mina: Bool, estado: Estado)
}
```

Quantas possíveis instâncias distintas existem de **Quadrado**? O campo **mina** pode assumir dois valores, e o campo **estado**, três.

Portanto, $2 \times 3 = 6$, que são os seis estados válidos que identificamos anteriormente.

Agora que temos uma representação adequada para um quadrado, podemos avançar e projetar uma função que determina como um quadrado ficará após a ação de um usuário. O usuário pode fazer uma ação para abrir um quadrado, adicionar uma bandeira ou remover uma bandeira.

Análise

- Determinar o novo estado de um quadrado a partir da ação do usuário.

Definição de tipos de dados

/// Uma ação do usuário no jogo.

```
type Acao {  
  Abrir  
  AdicionarBandeira  
  RemoverBandeira  
}
```

Exemplo - ação campo minado

Especificação

```
/// Atualiza o estado do quadrado *q* dada a *acao* do usuário... completar.  
fn atualiza_quadrado(q: Quadrado, acao: Acao) -> Quadrado
```

Qual é o campo do quadrado de entrada que pode mudar? Apenas o estado.

Do que depende a mudança do estado? Do estado atual e da ação.

Se o comportamento de uma função depende apenas de um valor enumerado, quantos exemplos precisamos colocar na especificação? Um para cada valor da enumeração.

A função que estamos projetando depende de apenas um valor enumerado? Não. Depende de dois, o valor do estado e o valor da ação.

Quantos exemplos precisamos nesse caso? Pelo menos $3 \times 3 = 9$ exemplos. Vamos fazer uma tabela para não esquecer de nenhum caso!

Exemplo - ação campo minado

estado/ação	abrir	adicionar	remover
aberto	-	-	-
fechado	aberto	fechado-com-bandeira	-
fechado-com-bandeira	-	-	fechado

```
check.eq(  
  atualiza_quadrado(Quadrado(False, Aberto), Abrir),  
  Quadrado(False, Aberto)  
) // q
```

```
check.eq(  
  atualiza_quadrado(Quadrado(False, Fechado), Abrir),  
  Quadrado(False, Aberto)  
) // Quadrado(..q, estado: Aberto)
```

```
// restante dos exemplos
```

Implementação

Se o comportamento de uma função depende apenas de um valor enumerado, qual é a estrutura inicial do corpo da função? Um caso para cada valor enumerado.

A função que estamos projetando depende de dois valores enumerados. Qual deve ser a estrutura inicial do corpo da função? Uma seleção de dois níveis, cada nível para um valor enumerado; ou; uma seleção com uma condição para cada par dos valores enumerados.

Exemplo - ação campo minado

```
fn atualiza_quadrado(q: Quadrado, acao: Acao) -> Quadrado {
  case q.estado {
    Aberto -> case acao {
      Abrir -> todo
      AdicionarBandeira -> todo
      RemoverBandeira -> todo
    }
    Fechado -> case acao {
      Abrir -> todo
      AdicionarBandeira -> todo
      RemoverBandeira -> todo
    }
    FechadoComBandeira -> case acao {
      Abrir -> todo
      AdicionarBandeira -> todo
      RemoverBandeira -> todo
    }
  }
}

fn atualiza_quadrado(q: Quadrado, acao: Acao) -> Quadrado {
  case q.estado, acao {
    Aberto, Abrir -> todo
    Aberto, AdicionarBandeira -> todo
    Aberto, RemoverBandeira -> todo
    Fechado, Abrir -> todo
    Fechado, AdicionarBandeira -> todo
    Fechado, RemoverBandeira -> todo
    FechadoComBandeira, Abrir -> todo
    FechadoComBandeira, AdicionarBandeira -> todo
    FechadoComBandeira, RemoverBandeira -> todo
  }
}
```

estado/ação	abrir	adicionar	remover
aberto	-	-	-
fechado	aberto	fechado-com-bandeira	-
fechado-com-bandeira	-	-	fechado

Se olharmos a tabela de exemplos, vamos notar que em apenas 3 casos precisamos atualizar o quadrado, então, não é necessário colocar explicitamente no código os 9 casos, podemos simplificar o código antes mesmo de escrevê-lo!

Exemplo - ação campo minado

```
/// Atualiza o estado do quadrado *q* dada a *acao* do usuário. A atualização é  
/// feita conforme a tabela a seguir, onde - significa que o quadrado permanece  
/// como estava.
```

```
///
```

```
/// | estado/ação          | abrir | adicionar          | remover |
```

```
/// |-----:|:-----:|:-----:|:-----:|
```

```
/// | aberto              | -     | -                  | -       |
```

```
/// | fechado             | aberto | fechado-com-bandeira | -       |
```

```
/// | fechado-com-bandeira | -     | -                  | fechado |
```

```
fn atualiza_quadrado(q: Quadrado, acao: Acao) -> Quadrado {
```

```
    case q.estado, acao {
```

```
        Fechado, Abrir -> Quadrado(..q, estado: Aberto)
```

```
        Fechado, AdicionarBandeira -> Quadrado(..q, estado: FechadoComBandeira)
```

```
        FechadoComBandeira, RemoverBandeira -> Quadrado(..q, estado: Fechado)
```

```
        _, _ -> q
```

```
    }
```

```
}
```

Unões

Projete uma função que produza uma mensagem sobre o estado de uma tarefa. Uma tarefa pode estar em execução, ter sido concluída em uma duração específica e com uma mensagem de sucesso, ou ter falhado com um código e uma mensagem de erro.

Como representar o estado de uma tarefa?

Vamos tentar uma estrutura.

Exemplo - estado tarefa

```
/// O estado de uma tarefa.  
type EstadoTarefa {  
    EstadoTarefa(  
        // True se a tarefa está em execução, False caso contrário.  
        executando: Bool,  
        // Em caso de sucesso  
        duracao: Int,  
        msg_sucesso: String,  
        // Em caso de erro  
        codigo_erro: Int,  
        msg_erro: String  
    )  
}
```

Qual é o problema dessa representação? Possíveis estados inválidos. O que significa

`EstadoTarefa(True, 10, "Ótimo desempenho", 123, "Falha na conexão")?`

Analizando a descrição do problema conseguimos separar o estado da tarefa em três casos:

- Em execução
- Sucesso, com uma duração e uma mensagem
- Erro, com um código e uma mensagem

Esses casos são excludentes, ou seja, se a tarefa se enquadra em um deles, não se deve armazenar informações sobre os outros (caso contrário, seria possível criar um estado inconsistente).

E como podemos expressar esse tipo de dado? Usando uma união de tipos.

Em uma **união**, os valores do tipo são divididos em casos, chamados **variantes**.

Cada variante tem o seu próprio **construtor** e pode ter os seus próprios campos.

Um valor do tipo pertence a exatamente uma variante.

Usamos uniões quando a informação é um entre vários casos excludentes, e cada caso pode ter informações próprias.

Uma enumeração é o caso particular em que nenhuma variante tem campos.

Definimos anteriormente um tipo de dado como um conjunto de possíveis valores. Agora vamos discutir qual é a relação entre a definição de tipos de dados e as operações com conjuntos.

- Os valores possíveis para um tipo definido por uma estrutura (**tipo produto**) são o produto cartesiano dos valores possíveis de cada um dos seus campos;
- Os valores possíveis para um tipo definido por uma união (**tipo soma**) são a união dos valores de cada variante do tipo.
- Chamamos de **tipo algébrico de dado** um tipo soma de tipos produtos.

Entender essa relação pode nos ajudar na definição dos tipos de dados, como foi para o quadrado do campo minado e como será para o estado da tarefa.

Algumas linguagens, como Rust e Python, têm construções separadas para estruturas e uniões.

A maioria das linguagens funcionais, incluindo o Gleam, tem apenas uma construção.

A forma geral para definição de tipos de dados em Gleam é

```
[pub] type NomeDoTipo {  
  Caso1([campo1:] Tipo1, [campo2:] Tipo2, ...])  
  Caso2([campo1:] Tipo1, [campo2:] Tipo2, ...])  
  ...  
}
```

<pre>[pub] type NomeDoTipo { Valor1 ... }</pre>	<pre>[pub] type NomeDoTipo { NomeDoTipo([campo1:] Tipo1, [campo2:] Tipo2, ...) }</pre>
---	--

Exemplo - estado tarefa

```
/// O estado de uma tarefa
type EstadoTarefa {
  // A tarefa está em execução
  Executando
  // A tarefa finalizou com sucesso.
  // Requer que duracao seja >= 0.
  Sucesso(duracao: Int, msg: String)
  // A tarefa finalizou com falha
  Erro(codigo: Int, msg: String)
}
```

```
> let estado: EstadoTarefa = Executando
> estado.msg
1 | estado.msg
  |           ^^^ This field does not exist

> fn duracao(estado: EstadoTarefa) {
  estado.duracao
}
2 | estado.duracao
  |           ^^^^^^^ This field does not exist
```

Então, como podemos acessar os campos!? Usando casamento de padrões com o `case`.

No **casamento de padrões**, comparamos um valor com um **padrão**, que descreve a forma esperada do valor.

Se o valor tem essa forma, o casamento tem sucesso e as variáveis do padrão são associadas às partes correspondentes do valor.

Um **padrão** pode ser

- uma variável, que casa com qualquer valor e o nomeia;
- o coringa `_`, que casa com qualquer valor e não o nomeia;
- um literal, que casa apenas com aquele valor;
- um construtor aplicado a **padrões**, que casa se o valor foi construído com esse construtor e se cada **padrão** interno casa com o campo correspondente;
- uma alternativa entre padrões, escrita com `|`, que casa se algum deles casar.

No **case**, o valor examinado é comparado com o padrão de cada caso, de cima para baixo, e vence o **primeiro** que casar. Por isso um padrão que casa com qualquer valor, como o `_`, só faz sentido no último caso.

Já usamos duas dessas formas antes de nomeá-las:

o `|`, no custo do tíquete,

```
case usuario {  
  Aluno | ServidorAte3 -> 5.0  
  ServidorMais3 | Docente -> 10.0  
  Externo -> 19.0  
}
```

e a vírgula, que examina mais de uma expressão, na atualização do quadrado.

```
case q.estado, acao {  
  Fechado, Abrir -> ...  
  _, _ -> q  
}
```

Exemplo - estado tarefa

```
/// O estado de uma tarefa
type EstadoTarefa {
  // A tarefa está em execução
  Executando
  // A tarefa finalizou com sucesso.
  // Requer que duracao seja >= 0.
  Sucesso(duracao: Int, msg: String)
  // A tarefa finalizou com falha
  Erro(codigo: Int, msg: String)
}
```

```
> /// Produz a duração ou -1 se não tem duração.
> fn duracao(estado: EstadoTarefa) -> Int {
  case estado {
    Sucesso(duracao, _) -> duracao
    _ -> -1
  }
}

> duracao(Executando)
-1
> duracao(Sucesso(10, "Recuperação exitosa. "))
10
> duracao(Erro(-23, "Arquivo não existente. "))
-1
```

Devolver `-1` quando não existe duração é uma solução ruim: estamos usando um `Int` válido para dizer “não tem valor”. Vamos resolver isso no capítulo **Funções totais**.

Agora podemos retornar e concluir o projeto.

Projete uma função que produza uma mensagem sobre o estado de uma tarefa. Uma tarefa pode estar em execução, ter sido concluída em uma duração específica e com uma mensagem de sucesso, ou ter falhado com um código e uma mensagem de erro.

Especificação

```
/// Produz uma string amigável para o usuário que descreve o *estado* de uma tarefa.  
fn mensagem(estado: EstadoTarefa) -> String
```

O exercício não é muito específico sobre a saída (o foco é no projeto de tipos de dados), por isso usamos a criatividade para definir a saída nos exemplos a seguir.

Exemplo - estado tarefa

Quantos exemplos são necessários? Pelo menos um para cada variante.

```
pub fn mensagem_examples() {  
    check.eq(  
        mensagem(Executando),  
        "A tarefa está em execução."  
    )  
    check.eq(  
        mensagem(Sucesso(12, "Os resultados estão corretos.")),  
        "Tarefa concluída (12s): Os resultados estão corretos."  
    )  
    check.eq(  
        mensagem(Erro(123, "Número inválido '12a'.")),  
        "A tarefa falhou (erro 123): Número inválido '12a'."  
    )  
}
```

Mesmo sem saber detalhes da implementação, podemos definir a estrutura do corpo da função com base apenas no tipo de dado, no caso, `EstadoTarefa`. São três casos:

```
fn mensagem(estado: EstadoTarefa) -> String {  
  case estado {  
    Executando -> todo  
  
    Sucesso(duracao, msg) -> todo  
  
    Erro(codigo, msg) -> todo  
  
  }  
}
```

Mesmo sem saber detalhes da implementação, podemos definir a estrutura do corpo da função com base apenas no tipo de dado, no caso, `EstadoTarefa`. São três casos:

```
fn mensagem(estado: EstadoTarefa) -> String {  
  case estado {  
    Executando ->  
      "A tarefa está em execução."  
    Sucesso(duracao, msg) ->  
      "Tarefa concluída (" <> int.to_string(duracao) <> "s): " <> msg  
    Erro(codigo, msg) ->  
      "A tarefa falhou (erro " <> int.to_string(codigo) <> "): " <> msg  
  }  
}
```

Uma estrutura é uma união com uma única variante. Então o seu construtor também é um padrão.

No **let**, o padrão precisa casar com **todos** os valores possíveis do tipo — é a mesma verificação de exaustividade do **case**. Com uma variante só e apenas variáveis nos campos, isso é garantido, e podemos desestruturar sem **case**:

```
> let p = Ponto(8, 2)
> // pela posição
> let Ponto(x, y) = p
> x
8
> y
2
```

```
> // pelo rótulo
> let Ponto(y: a, ..) = p
> a
2
> let Ponto(y:, ..) = p
> y
2
```

O **..** indica que os demais campos não interessam, e **y:** abrevia **y: y**. Um padrão com literal, como **Ponto(1, y)**, não é aceito no **let**, pois pode falhar.

Vimos que os tipos algébricos permitem modelar informações de forma mais precisa, tornando o programa mais confiável. Mas a utilidade deles aumenta muito quando a linguagem tem verificação estática de tipos.

Considere uma mudança nos requisitos: agora uma tarefa pode ficar em uma fila antes de começar a executar. Basta acrescentar a variante `Fila` em `EstadoTarefa`.

Mas, supondo que o programa use `EstadoTarefa` em vários lugares, como descobrir todos os pontos que precisam ser alterados?

O compilador faz isso por nós: cada `case` que deixou de ser exaustivo vira um erro *Inexhaustive patterns*, como o que vimos no início do capítulo.

Podemos usar tipos algébricos em outras linguagens? Sim, de fato, com o aumento do uso do paradigma funcional, muitas linguagens, mesmo algumas mais antigas como Java e Python, têm ganhado suporte a essa forma de definição de tipo de dados.

Vamos ver alguns exemplos.

```
@dataclass
class Executando:
    pass
```

```
@dataclass
class Sucesso:
    duracao: int
    msg: str
```

```
@dataclass
class Erro:
    codigo: int
    msg: str
```

```
EstadoTarefa = Executando | Sucesso | Erro
```

```
def mensagem(estado: EstadoTarefa) -> str:
    match estado:
        case Executando():
            return 'A tarefa está em execução'
        case Sucesso(duracao, msg):
            return f'A tarefa finalizou com sucesso ({duracao}s): {msg}'
        case Erro(codigo, msg):
            return f'A tarefa falhou (erro {codigo}): {msg}'
```

Aqui, usamos casamento de padrões para decompor cada tipo produto em seus componentes.

```
pub enum EstadoTarefa {  
    Executando,  
    Sucesso(u32, String),  
    Erro(u32, String),  
}  
  
pub fn mensagem(estado: &EstadoTarefa) -> String {  
    match estado {  
        EstadoTarefa::Executando =>  
            "A tarefa está em execução".to_string(),  
        EstadoTarefa::Sucesso(duracao, msg) =>  
            format!("A tarefa finalizou com sucesso ({duracao}s): {msg}"),  
        EstadoTarefa::Erro(codigo, msg) =>  
            format!("A tarefa falhou (erro {codigo}): {msg}"),  
    }  
}
```

Usamos novamente casamento de padrões para decompor **Sucesso** e **Erro** em seus componentes.

```
sealed interface EstadoTarefa permits Executando, Sucesso, Erro {}  
record Executando() implements EstadoTarefa {}  
record Sucesso(int duracao, String msg) implements EstadoTarefa {}  
record Erro(int codigo, String msg) implements EstadoTarefa {}  
  
static String mensagem(EstadoTarefa estado) {  
    return switch (estado) {  
        case Executando e ->  
            "A tarefa está em execução";  
        case Sucesso(int duracao, String msg) ->  
            String.format("A tarefa finalizou com sucesso (%ds): %s", duracao, msg);  
        case Erro(int codigo, String msg) ->  
            String.format("A tarefa falhou (erro %d): %s", codigo, msg);  
    };  
}
```

A [JEP 405](#), disponível desde o Java 21, permite o uso de padrões para decompor registros.

Revisão

Quais são as duas diretrizes para o projeto de tipos de dados?

- Torne os valores válidos representáveis.
- Torne os valores inválidos irrepresentáveis.

Quais são as duas formas de definir tipos algébricos e quando usar cada uma?

- Estruturas (tipo produto), quando dois ou mais itens de informação juntos descrevem uma entidade;
- Uniões e enumerações (tipo soma), quando a informação é um entre vários casos excludentes.

O que é casamento de padrões?

- É comparar um valor com um padrão, que descreve a forma esperada do valor; se o valor tem essa forma, as variáveis do padrão são associadas às partes correspondentes.

Quantos valores distintos tem um tipo produto? E um tipo soma?

- No tipo produto, o produto da quantidade de valores de cada campo; no tipo soma, a soma da quantidade de valores de cada variante.

Como um tipo soma com N casos guia o projeto de uma função que o recebe como entrada?

- Ele sugere pelo menos N exemplos, um para cada caso, e um corpo com uma análise de N casos.

E se a função recebe dois valores de tipos soma, com N e M casos?

- Pelo menos $N \times M$ exemplos; uma tabela ajuda a não esquecer nenhum.

O que acontece quando acrescentamos uma variante a um tipo soma?

- Cada **case** que deixou de ser exaustivo vira erro de compilação, e assim o compilador encontra os pontos do programa que precisam ser alterados.

Referências

Básicas

- [Tipos de dados em Gleam](#)
- [Vídeo Making Impossible States Impossible](#)

Complementares

- [Expression problem](#)